Appendix

1 Correctness Sketch

Let $\mathrm{Sol}(S)$ denote the intended abstract semantic space of an SDL specification S. Taking the ASP mapping Π_S^{ASP} as the reference semantics, we provide evidence that the CP mapping $\mathcal{M}_S$ yields consistent interpretations. Specifically, for any assignment to the guessed records, we argue that an interpretation valid under the non-monotonic ASP semantics has a natural structural counterpart that satisfies the monotonic CP constraints (with scalarized optimization), up to projection on the records mentioned in `show` directives. The argument proceeds by structural induction on the SDL instructions.

1. **Guess**: Both mappings enumerate equivalent Cartesian domains. The total and finite nature of the CP decision variables natively covers the exact search space generated by the ASP choice rules.

2. **Define (non-recursive)**: For each non-recursive `define` instruction, the mapping generates a biconditional constraint $R(\mathbf{x}) \leftrightarrow \bigvee_i B_i(\mathbf{x})$ across all defining rules. This construction directly implements Clark's Completion: the forward direction ($\leftarrow$) encodes the supporting rules, while the backward direction ($\rightarrow$) enforces minimality, ruling out any truth assignment not justified by an explicit rule body. Since non-recursive instructions induce no positive cycles in the dependency graph, the program is by definition tight. By Fages' Theorem [1], the models of Clark's Completion coincide precisely with the stable models for tight programs. Therefore, the biconditional encoding natively produces the same derived facts as the ASP back-end, including correct handling of Negation as Failure within rule bodies, which is mapped to classical negation under the Domain Closure Assumption (see the *Deny* paragraph below).

3. **Define (recursive)**: For non-tight programs containing positive cycles, Clark's Completion alone is insufficient to characterise the stable models: classical logic admits *unfounded sets*, i.e., sets of atoms that mutually support each other without any grounded base case, and therefore evaluate to true under the completion despite having no well-founded derivation. The rank-based encoding rules these out by construction. For each positively recursive record R, the mapping introduces a companion integer

array $\text{rank}_R : D_R \to \{0, \ldots, |D_R|\}$ and posts the acyclicity implication:

$$R(\mathbf{x}) \;\to\; \bigvee_{\mathbf{y}} \Big(B(\mathbf{x}, \mathbf{y}) \;\wedge\; \text{rank}_R[\mathbf{x}] > \text{rank}_R[\mathbf{y}] \Big).$$

Suppose, for contradiction, that a non-empty unfounded set $U \subseteq D_R$ exists in a solution; that is, every atom $R(\mathbf{x})$ with $\mathbf{x} \in U$ is true, yet no atom in U has a supporting derivation whose body is grounded outside U. By the implication above, each $\mathbf{x} \in U$ must be supported by some $\mathbf{y} \in U$ with $\text{rank}_R[\mathbf{x}] > \text{rank}_R[\mathbf{y}]$. This induces a strictly decreasing chain within U. Since U is finite and the rank domain is bounded below by 0, no such infinite descent exists ; a contradiction. Therefore every true atom in R must ultimately be traceable to a base case outside any cycle, which aligns with the well-foundedness condition of stable model semantics. This argument corresponds to the classical level mapping characterisation [3] of unfounded sets [2], instantiated here over the finite CP domain; the connection to stable models for tight programs then follows by Fages' Theorem [1]. Conversely, every stable model A yields a valid rank assignment by setting $\mathit{rank}_R[\mathbf{x}]$ to the derivation depth of $R(\mathbf{x})$ in A; well-foundedness of stable models guarantees this satisfies the acyclicity implication.

4. **Deny**: Both mappings enforce the same integrity prohibitions. The critical subtlety lies in Negation as Failure: in ASP, $\neg_{\text{naf}} R(\mathbf{x})$ holds whenever $R(\mathbf{x})$ cannot be derived, reflecting the Closed World Assumption. In the CP mapping, this is rendered as $\neg\exists\, \mathbf{y} \in D_R : \varphi(\mathbf{x}, \mathbf{y})$. The soundness of this translation rests on the **Domain Closure Assumption (DCA)**: because every domain D_R generated by $\mathcal{M}(S)$ is strictly finite and explicitly bounded, the exhaustive enumeration of D_R is complete. Consequently, the failure to find a witness $\mathbf{y}$ satisfying φ within D_R is treated as consistent with classical falsity; there is no open-world extension in which $R(\mathbf{x})$ could become true. The DCA is guaranteed by construction: the compiler never generates unbounded or parametric domains, ensuring that classical negation over D_R faithfully captures the non-monotonic NAF semantics of the original SDL specification.

5. **Weak Constraints**: SDL soft denials are stratified into priority levels $1, \ldots, k$, with higher levels taking lexicographic precedence. The CP mapping scalarizes this hierarchy by assigning a static multiplier μ_L to each level, defined inductively as:

$$\mu_1 = 1, \qquad \mu_L = \mu_{L-1} \cdot \big(\text{MaxCost}_{L-1} + 1\big),$$

where

$$\text{MaxCost}_L = \sum_{c \in \mathcal{W}_L} \sum_{\mathbf{x} \in \text{Dom}(c)} C_c(\mathbf{x})$$

is the worst-case cumulative penalty at level L. We argue by induction on L that this scaling preserves the global lexicographic ordering. The

base case is trivial: at level 1, $\mu_1 = 1$ and no lower level exists. For the inductive step, assume that for all levels $1, \ldots, L-1$ the scaling correctly preserves lexicographic dominance. By construction:

$$\mu_L = \mu_{L-1} \cdot \big(\text{MaxCost}_{L-1} + 1\big) > \text{MaxCost}_{L-1}.$$

This means that a single unit of improvement at level L contributes μ_L to the scalar objective, which strictly exceeds the maximum total penalty achievable across all constraints at level $L-1$ combined. Moreover, by unrolling the induction one can verify that

$$\mu_L = \prod_{j=1}^{L-1} (\text{MaxCost}_j + 1)$$

from which the global dominance follows directly:

$$\mu_L \; > \; \sum_{j=1}^{L-1} \mu_j \cdot \text{MaxCost}_j.$$

Dominance is therefore global and transitive across the entire priority hierarchy. Consequently, no accumulation of penalties at any combination of lower levels can compensate for a higher-level violation, and the minimization of:

$$\text{total\_penalty} = \sum_{L=1}^{k} \mu_L \cdot p_L$$

is mathematically isomorphic to lexicographic minimization over $(p_k, \ldots, p_1)$. The induction closes at level k, establishing that the scalarized CP objective faithfully reproduces the priority structure of ASP weak constraints across all levels.

References

[1] F. Fages, *Consistency of Clark's completion and existence of stable models*, J. Methods Log. Comput. Sci. 1 (1994) 51–60.

[2] A. Van Gelder, K. A. Ross, J. S. Schlipf, *The well-founded semantics for general logic programs*, J. ACM 38 (1991) 620–650.

[3] K. R. Apt, H. A. Blair, A. Walker, *Towards a theory of declarative knowledge*, in: Foundations of Deductive Databases and Logic Programming, Morgan Kaufmann, 1988, pp. 89–148.